HCLTech × UBC AI / ML Hackathon

Student Participation Guidelines · July 6–17, 2025

HACKATHON TIMELINE

Timeline	Phase	Key Activities
Week 1 (Jul 6–7)	Kickoff + Use Case Release	Team registration, use case selection, initial plan
Week 2 (Jul 8–14)	Build Period	MVP/prototype, mentoring sessions, architecture check-in
Week 3 (Jul 15–16)	Refinement & Submission	Stabilise demo, code cleanup, final submission (16 Jul)
Finale (Jul 17)	Presentation Day	Live demo + code walkthrough + Q&A · 2–5 PM PST · 12 min per team (8 min + 4 min Q&A)
Post Finale	Certificates	Issued within 7–10 business days — confirm full name and email after the event

USE CASE & PROJECT GUIDELINES

- Each team selects one use case to work on for the entire hackathon. Email your selection to sumit-k@hcltech.com.
- Team size: 4–5 members per team.
- Build a working solution — prototype/MVP quality is expected by Week 2.
- Your solution must address the problem statement and all mandatory deliverables in the selected use case.
- Datasets can be downloaded from the internet or self-generated as described in each use case.

GITHUB & CODE SUBMISSION — MANDATORY

- All code must be committed to a GitHub repository throughout the hackathon (not just on the last day).
- Share repository access with GitHub usernames: sksaha-hcl and deepaksing-hcl before Day 3.
- Maintain a clean, well-structured repository with meaningful commit messages.
- Include a README.md with setup instructions and how to run the project.
- Include your architecture diagram and presentation deck in the repository.
- Repositories not shared by 11:59 PM PST on 16th July will be considered incomplete.

PRESENTATION DAY — 17TH JULY, 2–5 PM PST

- Presentation will be conducted in person (venue shared by 10th July). Remote participation is welcome.
- Each team has 12 minutes total: 8-minute presentation + 4-minute Q&A from HCLTech SMEs and leadership.
- Presentation must include: solution design overview, live or recorded demo, and brief code walkthrough.
- At least one team member must present. Remote members may join via video conference.

TIME COMMITMENT

There is no fixed daily hour requirement. The more effort your team invests, the stronger your project will be. Plan your own schedule across the 12 active days (Jul 6–16).

MENTORING & SUPPORT

- HCLTech SMEs will hold office hours during Week 2 (2–3 slots) for architecture and scoping feedback.
- Email any questions to the mentors listed in your registration confirmation.
- API usage and costs: Teams should use free-tier API access or open-source models (e.g. Ollama, LLaMA). HCLTech will not reimburse API costs unless explicitly pre-approved by your assigned mentor before incurring charges.

CERTIFICATES & REWARDS

- All participants will receive an HCLTech Hackathon Participation Certificate.
- Top-performing teams will be recognised at the finale and in HCLTech internal communications.
- Certificates issued within 7–10 business days after the finale. Confirm your full name and email after the event.

INTELLECTUAL PROPERTY

All code developed during the hackathon remains the intellectual property of the student team. By participating, teams grant HCLTech a non-exclusive licence to showcase submitted work for internal demonstration and educational purposes.

HCLTech

RETAIL · Use Case #01

Customer Sentiment Analysis & GenAI Insights Hub

Transform thousands of unread customer reviews and social posts into clear, actionable product and service intelligence — in real time.

#01

PROBLEM STATEMENT

Retailers receive thousands of reviews, social mentions, and support tickets every day — but analyse only a fraction manually. This unstructured feedback contains valuable signals: quality issues, feature requests, competitor comparisons, and loyalty indicators. Teams will build an NLP-powered sentiment pipeline that extracts actionable insights across channels and uses GenAI to auto-generate weekly product intelligence reports for retail managers.

BUSINESS IMPACT

93%	85%	NPS +15	ABSA
Purchases influenced by reviews	Feedback never analysed	Avg. improvement with action	Key NLP technique

WHAT YOU'LL BUILD

- Build a sentiment classification pipeline (Positive / Negative / Neutral) using BERT or RoBERTa fine-tuned on retail review data
- Implement Aspect-Based Sentiment Analysis (ABSA) — extract sentiment per product feature (price, quality, delivery, service)
- Apply topic modelling (LDA or BERTopic) to discover emerging customer themes and surface trend shifts over time
- Build a GenAI weekly summary engine: top complaints, positive trends, and recommended actions for retail managers
- Create a real-time insights dashboard with sentiment trend charts, ABSA breakdown, and a configurable alert system

SUGGESTED TECH STACK

Python · HuggingFace Transformers · BERT / RoBERTa · BERTopic · Claude/OpenAI API · Streamlit / Plotly · NLTK / spaCy · FastAPI · SQLite

SUGGESTED DATASETS

- Kaggle: Amazon Customer Reviews Dataset (multi-category, millions of rows — use a single category subset)
- Kaggle: Yelp Dataset (restaurant and retail reviews with star ratings)
- HuggingFace: tweet_eval benchmark dataset (sentiment subset, 45K labelled tweets — replaces direct Twitter scraping)

12-DAY SPRINT TIMELINE (JUL 6–16)

Days 1–2	Days 3–5	Days 6–8	Days 9–10	Days 11–12
Data prep, EDA & baseline sentiment model	BERT/RoBERTa fine-tuning + ABSA pipeline	Topic modelling & trend detection	GenAI report generator + dashboard build	Integration testing, live demo prep & deck

DELIVERABLES

1. Sentiment classification model achieving F1 > 0.75 on held-out test set, plus ABSA output per product category
2. Topic model with labelled themes and their trend over time visualised on the dashboard
3. Auto-generated weekly product intelligence report using GenAI with source citations
4. Real-time insights dashboard with filtering by product, date range, and review channel
5. Live demo: analyse a new product's first 200 reviews during the presentation

JUDGING CRITERIA (TOTAL: 100%)

25% Model Accuracy (F1)	**25%** ABSA Quality & Insight Depth	**25%** GenAI Report Quality	**15%** Dashboard Usability & Design	**10%** Live Demo Quality

BONUS CHALLENGES (OPTIONAL — EXTRA CREDIT)

+ Add competitor brand sentiment comparison displayed side-by-side
+ Support multilingual sentiment analysis (Spanish, Hindi, or French)
+ Build a real-time streaming ingestion pipeline using a public HuggingFace dataset loader
+ Alert system: detect viral negative sentiment spike within a configurable time window

HCLTech

AMS / OBSERVABILITY · Use Case #02

AI-Driven Log Analysis & Root Cause Investigation

Move from a customer complaint to an evidence-backed root cause in minutes — using AI agents that correlate logs, trace transactions, and validate findings in source code.

#02

PROBLEM STATEMENT

Production systems generate thousands of log lines per minute. When a customer-facing incident occurs, engineers must manually sift through logs across multiple tools (Dynatrace, Splunk, ELK), correlate transaction traces, and identify the root cause — often under strict SLA pressure. This process is slow, error-prone, and relies heavily on individual expertise. Teams will build an AI-powered investigation assistant that can move from a customer complaint to an evidence-based root cause within minutes, not hours.

BUSINESS IMPACT

4–8 h	$5,600/min	70%	3+ tools
Avg. RCA time per incident	Cost of enterprise downtime	Incidents that recur w/o proper RCA	Searched per incident avg.

WHAT YOU'LL BUILD

- Log ingestion pipeline: parse and normalise multi-source synthetic logs (application, transaction, monitoring) into a unified queryable format
- LLM + RAG investigation engine: given a customer complaint or incident ID, autonomously reconstruct the full transaction journey and identify the failure point with cited log evidence
- Source code correlation module: link the identified failure to the relevant exception handler or business rule in a public Git repository using GitPython
- Structured RCA report generator: root cause statement, short-term fix, and long-term architectural recommendation — every claim backed by a log line or code reference
- Address at least ONE of three scenarios: Unhandled Application Exception (A), Eligibility Decision Failure (B), or Silently Failed Financial Transaction (C)

SUGGESTED TECH STACK

Python · LangChain · Claude / GPT API · Elasticsearch / local ELK · GitPython · Streamlit / FastAPI · Faker (synthetic log generation) · OpenTelemetry (optional)

SUGGESTED DATASETS

- Self-generated synthetic application logs using Python Faker + custom log templates (application, transaction, KYC/profile schemas provided in starter kit)
- Public GitHub repository for source code analysis — use any open-source Flask, FastAPI, or Spring Boot app
- Kaggle: Server and Application Log Datasets (for log format reference and anomaly pattern examples)

12-DAY SPRINT TIMELINE (JUL 6–16)

Days 1–2	Days 3–5	Days 6–8	Days 9–10	Days 11–12
Synthetic log generation, scenario selection & architecture design	Log ingestion pipeline + LLM investigation engine	RCA module + Git source code correlation	RCA report generator + transaction journey diagram	Scenario demo end-to-end, presentation deck & code polish

DELIVERABLES

1. Working AI/LLM prototype that ingests synthetic logs and outputs a structured RCA report for at least one scenario
2. Customer journey or transaction flow diagram reconstructed automatically by the AI agent
3. Root Cause Statement with evidence cited from logs and source code (file + line reference)
4. Short-term fix recommendation and long-term architectural improvement suggestion
5. 5-minute live or recorded demo + 5–7 slide presentation deck

JUDGING CRITERIA (TOTAL: 100%)

30%	25%	20%	15%	10%
Investigation Accuracy & Evidence	RCA Quality & Code Validation	AI & Observability Tool Use	Fix Recommendations	Presentation & Documentation

BONUS CHALLENGES (OPTIONAL — EXTRA CREDIT)

+ Auto-generated incident summaries requiring zero manual input from the engineer
+ RAG over source code repository — query the codebase in natural language
+ Intelligent runbooks that auto-suggest remediation steps based on the identified root cause
+ Automated log correlation across two or more simulated observability tool outputs

HCLTech

DEVELOPER TOOLING / CODE INTELLIGENCE · Use Case #03

Source Code Knowledge Graph Assistant (CodeIQ)

Parse a real codebase into a queryable knowledge graph — then let any developer ask plain-English questions about code structure, component impact, and test coverage.

#03

PROBLEM STATEMENT

Large React / React Native codebases span thousands of files. Developers waste significant time manually tracing how UI screens, components, hooks, APIs, and feature flags connect. New team members have no structured way to explore the codebase, and traditional keyword search returns text matches — not relationships. Build CodeIQ: an AI assistant that parses source code into a knowledge graph and answers plain-English questions about structure and impact. NOTE: This is the most technically challenging use case. Recommended for teams with TypeScript and graph database experience. An MVP demonstrating 3 of the 5 entity types is fully acceptable.

BUSINESS IMPACT

19%	3 h avg	100 K+	Expert
Dev time lost to codebase navigation	To trace a single unfamiliar flow	Lines in a typical enterprise React app	Recommended experience level

WHAT YOU'LL BUILD

- Code parser using ts-morph or the TypeScript Compiler API to extract at least 3 entity types (e.g. Screens, Components, Hooks) and their typed relationships from the target codebase
- Knowledge graph stored in Neo4j or NetworkX (MVP) with nodes for entities and typed edges for relationships: renders→, calls→, depends on→, covers→
- Vector search index using ChromaDB or LanceDB enabling semantic natural-language queries over code entities and their descriptions
- LLM-powered Q&A agent (LangChain) that answers at least 5 natural-language questions about code structure with source evidence — file path and relevant code snippet
- Simple chat UI or console interface returning a Confidence level (High / Medium / Low) with every answer

SUGGESTED TECH STACK

Python / Node.js · ts-morph / TypeScript Compiler API · Neo4j (or NetworkX for MVP) · ChromaDB / LanceDB · LangChain / OpenAI · SQLite FTS5 · React chat UI or console

SUGGESTED DATASETS

- Primary: bluesky-social/social-app (open-source React Native codebase on GitHub, ~100K lines) — or any public TypeScript React project under 500 files for MVP scope
- Self-generated Q&A evaluation set — create 10 natural-language question + expected-answer pairs before building, to measure answer quality
- Optional: a second smaller repo to demonstrate the parser generalises beyond one codebase

12-DAY SPRINT TIMELINE (JUL 6–16)

Days 1–2	Days 3–5	Days 6–8	Days 9–10	Days 11–12
Parser setup, entity schema design & target codebase selection	Knowledge graph construction (3+ entity types + relationship edges)	Vector search + LLM Q&A agent integration	Chat interface + confidence scoring + answer evaluation (10 test questions)	Demo polish, 5 live questions answered, architecture diagram & deck

DELIVERABLES

1. Working chat interface backed by a real knowledge graph (partial codebase coverage is acceptable for MVP — state what is covered)
2. Parsed graph with 3+ entity types and relationship edges extracted from the target codebase
3. Answers to at least 5 natural-language test questions with source evidence (file path + code snippet) for each
4. Confidence level (High / Medium / Low) returned for every answer, with a brief rationale
5. Architecture diagram showing parser → graph → vector search → LLM → response pipeline + 5-minute demo + 5–7 slide deck

JUDGING CRITERIA (TOTAL: 100%)

30%	25%	20%	15%	10%
Graph Accuracy & Coverage	Answer Relevance & Quality	Evidence Grounding (file + snippet)	Query Performance & Confidence	Presentation & Diagram

BONUS CHALLENGES (OPTIONAL — EXTRA CREDIT)

+ Index all 5+ entity types (Screens, Components, Hooks, Services/APIs, Feature Flags, Tests)
+ Support impact analysis: 'What breaks if session restore changes?' — trace all affected nodes
+ Extend the parser to a second language or framework (e.g. Angular or Java Spring)
+ Feature flag impact tracing: show which screens and components each config key controls

BANKING · Use Case #04

Intelligent Banking Assistant (RAG Chatbot)

A GenAI-powered chatbot grounded in real banking documents — answering customer questions accurately, safely, and without hallucination.

#04

PROBLEM STATEMENT

Bank call centres handle millions of repetitive enquiries daily — balance checks, loan eligibility, fee explanations, and product comparisons. GenAI chatbots powered by Retrieval-Augmented Generation (RAG) can resolve 60–80% of these queries instantly. The core challenge: building a reliable, hallucination-free bot grounded in official banking documents and policies — not model imagination.

BUSINESS IMPACT

60–80%	$8/call	24/7	< 3 s
Queries auto-resolved	Human agent cost	Availability	Target response time

WHAT YOU'LL BUILD

- RAG pipeline: ingest banking PDFs, FAQs, and policy documents into a vector store with chunking and metadata tagging for accurate retrieval
- LLM-powered response generation with source document citations shown to the user for every factual answer
- Intent classifier to route complex, sensitive, or out-of-scope queries gracefully to a human escalation path
- Conversational memory enabling coherent multi-turn interactions — the bot remembers context within a session
- Chat UI showing a confidence score, source document links, and an escalation button for human hand-off

SUGGESTED TECH STACK

Python · LangChain / LlamaIndex · Claude/GPT API · ChromaDB / Pinecone · React / Streamlit · FastAPI · HuggingFace Embeddings · PyPDF2

SUGGESTED DATASETS

- Public bank FAQ documents (HDFC, SBI, Wells Fargo — available on their official websites)
- RBI / Federal Reserve public policy PDFs (freely available for download from official regulatory websites)
- Synthetic Q&A pairs — generate 50+ test question/answer pairs to evaluate chatbot accuracy and hallucination rate

12-DAY SPRINT TIMELINE (JUL 6–16)

Days 1–2	Days 3–5	Days 6–8	Days 9–10	Days 11–12
Scope chatbot intents & design the knowledge base structure	Document ingestion, chunking strategy & RAG pipeline build	LLM integration, intent classifier & safety guardrails	Chat UI build, evaluation framework & hallucination testing	Multi-turn scenario testing, demo polish & presentation deck

DELIVERABLES

1. Functional RAG chatbot handling 20+ defined banking intents across at least three product areas
2. Knowledge base with 50+ pages of indexed banking documents and a documented chunking strategy
3. Evaluation report: accuracy on 20 test questions, hallucination count, and average response quality score
4. Human escalation routing logic with graceful fallback for sensitive or ambiguous queries
5. Live demo of 3 complex multi-turn customer scenarios + architecture diagram

JUDGING CRITERIA (TOTAL: 100%)

30%	25%	20%	15%	10%
Accuracy & Groundedness (no hallucination)	RAG Pipeline Architecture Quality	UX, Safety & Guardrail Design	Business Application Breadth	Live Demo Quality

BONUS CHALLENGES (OPTIONAL — EXTRA CREDIT)

+ Add multilingual support (Hindi, Spanish, or French) using multilingual embedding models
+ Implement voice input and output interface using browser speech APIs
+ Build guardrails to detect and block financially harmful or misleading outputs
+ Add session memory with user profile personalisation (preferred account type, past queries)

HCLTech

RETAIL · Use Case #05

Personalized Product Recommendation Engine

Turn anonymous shoppers into loyal buyers — recommending the right product to the right person at exactly the right moment.

#05

PROBLEM STATEMENT

35% of Amazon's revenue comes from its recommendation engine. Retailers that display generic product grids miss the enormous revenue potential of personalisation. Teams will build a hybrid recommendation system combining collaborative filtering (what similar users bought) with content-based filtering (product attributes) — and optionally a GenAI layer for conversational product discovery.

BUSINESS IMPACT

35%	10–30%	3×	Cold Start
Amazon revenue from recs	Avg. conversion lift	Typical cart size increase	Key technical challenge

WHAT YOU'LL BUILD

- Collaborative filtering model using matrix factorisation (ALS or SVD) trained on user–item interaction data
- Content-based model using product metadata, TF-IDF, and sentence embeddings to capture item similarity
- Hybrid ensemble combining both signals with a learning-to-rank layer for final recommendation ordering
- Cold-start strategy for brand new users (demographic-based) and brand new products (content-only fallback)
- Product discovery UI with a 'Recommended for You' feed, explainability text ('Because you viewed X'), and A/B simulation

SUGGESTED TECH STACK

Python · Surprise / LightFM · Scikit-learn · Sentence-Transformers · FAISS · FastAPI · React / Streamlit · Pandas · MLflow

SUGGESTED DATASETS

- Kaggle: H&M Fashion Recommendations Dataset (customer purchase history + product metadata)
- Amazon Product Reviews (various categories — use a single category for focused modelling)
- MovieLens 1M or 10M (proxy dataset for collaborative filtering algorithm development)

12-DAY SPRINT TIMELINE (JUL 6–16)

Days 1–2	Days 3–5	Days 6–8	Days 9–10	Days 11–12
EDA, interaction matrix construction & baseline collaborative filter	Content-based model + hybrid ensemble build	Cold-start strategy + recommendation API	Product discovery UI + A/B simulation + explainability layer	Offline metric evaluation, demo polish & presentation deck

DELIVERABLES

1. Hybrid recommendation system with NDCG, MAP, and Hit Rate metrics documented on a held-out test set

2. Cold-start solution demonstrating strategy for both new users and new items with example outputs
3. Product discovery UI with a personalised homepage feed and 'Recommended because…' explanations
4. A/B simulation comparing personalised recommendations vs. a popularity-based random baseline
5. Architecture diagram showing data flow from user interaction to ranked recommendation output

JUDGING CRITERIA (TOTAL: 100%)

30%	25%	20%	15%	10%
Recommendation Quality (NDCG / MAP)	Hybrid Architecture Design	Cold Start Strategy	UX & Business Impact	Presentation

BONUS CHALLENGES (OPTIONAL — EXTRA CREDIT)

+ Add real-time session-based re-ranking using in-session clickstream signals
+ Build a GenAI conversational shopping assistant layer on top of the recommendation engine
+ Optimise recommendations for diversity and serendipity (penalise overly similar results)
+ Use multi-modal embeddings combining product image features with text metadata

BANKING / FINTECH · Use Case #06

AI-Driven KYC Onboarding & Compliance Engine

Dynamically adapts the entire verification flow — document checks, biometrics, sanctions screening, and fraud detection — to each user's jurisdiction, delivering sub-60-second compliant onboarding at scale.

#06

PROBLEM STATEMENT

FinTech companies expanding internationally face a deeply fragmented regulatory landscape: each country demands distinct Customer Due Diligence (CDD) templates, varied document types, and unique verification flows. A one-size-fits-all onboarding process results in high user drop-off rates (40–60%), costly manual compliance reviews, and vulnerability to sophisticated synthetic identity fraud. The challenge is to build an AI-powered onboarding engine that dynamically adapts its verification steps to the user's jurisdiction — delivering a seamless, sub-60-second experience while remaining fully audit-ready and compliant.

BUSINESS IMPACT

40–60% Avg. KYC onboarding drop-off	**$25–50** Cost of one manual KYC review	**< 60 s** Target automated onboarding time	**$3.1 B** Global KYC market size

WHAT YOU'LL BUILD

- Dynamic Regulatory Engine — an AI orchestrator that detects the user's jurisdiction and adjusts the onboarding flow, required CDD fields, document types, and verification method (eKYC vs. Video KYC) in real time
- eKYC document pipeline — extract and validate data from government-issued IDs using OCR; perform biometric face-matching comparing a pre-collected synthetic selfie to the extracted ID photo with a liveness confidence score. Note: use pre-collected synthetic selfie + ID photo pairs for the demo; real-time camera capture is a bonus extension.
- Sanctions & PEP screening — screen each applicant against simplified OFAC, UN, and EU blocklists; flag Politically Exposed Persons (PEPs) and sanctioned entities before onboarding completes
- Adverse Media OSINT agent — deploy a GenAI agent to scan public news sources and open databases for negative coverage linked to the applicant name or entity, surfacing reputational risk
- Behavioural fraud scorer — analyse metadata signals (typing cadence, device fingerprint, session timing) to generate a real-time synthetic identity risk score before the user submits the form

SUGGESTED TECH STACK

Python · OpenCV / face_recognition · LangChain / LangGraph · Claude / GPT API · FastAPI · React / Next.js · Pandas · Docker · Faker (test identity generation)

SUGGESTED DATASETS

- Faker library: generate synthetic identity profiles with country-specific CDD fields for at least two fictional jurisdictions
- OFAC SDN List (public, downloadable at ofac.treas.gov) and OpenSanctions dataset for PEP and sanctions screening
- Self-designed mock CDD templates for two fictional countries with distinct regulatory rules (e.g. one requiring biometric + NFC, one requiring Video KYC only)

12-DAY SPRINT TIMELINE (JUL 6–16)

Days 1–2	Days 3–5	Days 6–8	Days 9–10	Days 11–12
Architecture design, 2-country CDD rule definition & mock identity data setup	eKYC pipeline: document OCR, field extraction & biometric face-matching	Dynamic Regulatory Engine + sanctions / PEP screening integration	GenAI OSINT adverse media agent + behavioural fraud scoring module	End-to-end onboarding UI, fraud sandbox demo, pitch deck & code polish

DELIVERABLES

1. Working onboarding prototype for 2 fictional countries with distinct CDD flows, document requirements, and triggered verification paths (eKYC vs. Video KYC)
2. eKYC pipeline demonstration: document OCR extraction, field validation, and biometric face-match with liveness confidence score
3. Sanctions / PEP screening with a live alert triggered on a flagged synthetic applicant during the demo
4. Fraud sandbox: system detects a synthetic identity attempt using behavioural signals or OSINT adverse media data
5. Architecture diagram showing secure data flow between frontend, AI agents, and APIs + 3-minute pitch deck with business value and compliance case

JUDGING CRITERIA (TOTAL: 100%)

30%	25%	20%	15%	10%
Dynamic regulatory adaptation & eKYC accuracy	Fraud detection quality (OSINT + behavioural)	Compliance coverage — sanctions/PEP & audit trail	UX & onboarding flow — sub-60-second demo	Business pitch & architecture clarity

BONUS CHALLENGES (OPTIONAL — EXTRA CREDIT)

+ Build a Video KYC fallback module triggered automatically when eKYC confidence falls below a configurable threshold
+ Simulate NFC chip reading for ePassport verification as an additional identity assurance layer
+ Extend regulatory support to 3+ countries or regions (e.g. UK FCA, MAS Singapore, EU AMLD6)
+ Add continuous monitoring: re-screen onboarded customers automatically when new adverse media is detected

BANKING / FINTECH · Use Case #07

AI-Powered Financial Crime Investigation Agent

End-to-end agentic AI that autonomously gathers evidence, detects money-laundering typologies, and drafts compliant Suspicious Activity Reports — compressing 4–8 hours of analyst work to under 30 minutes.

#07

PROBLEM STATEMENT

Financial crime compliance teams spend 4–8 hours per case manually correlating data across core banking systems, KYC records, and sanctions lists — all fragmented across disconnected tools. With a 30% false-positive alert rate, a strict 72-hour SAR filing deadline, and growing regulatory scrutiny, analysts face mounting backlogs and inconsistent outcomes. An agentic AI system that autonomously gathers evidence, detects known laundering typologies, scores risk with a clear reasoning chain, and drafts a submission-ready Suspicious Activity Report can transform this workflow.

BUSINESS IMPACT

4–8 h	72 h	30%	$2 T+
Avg. time per case today	Regulatory SAR deadline	Alert false-positive rate	Estimated annual money laundering

WHAT YOU'LL BUILD

- Agentic evidence pipeline — given a flagged transaction ID, autonomously query synthetic KYC records, account transaction history, and a simplified sanctions list using LLM tool-calling
- Risk scoring engine — output a 0–100 fraud probability score backed by a plain-English reasoning chain; no black-box verdicts
- Typology detector — identify at least one known money-laundering pattern: smurfing, layering, or round-tripping within a connected transaction graph
- SAR narrative generator — map investigation findings to mandatory FinCEN/FCA fields; every factual claim linked to a source; detect and flag missing mandatory fields before finalising
- Analyst Q&A interface — multi-turn natural-language chat over the live case (e.g. 'Has this entity appeared on any prior SAR?'), with source citations on every answer

SUGGESTED TECH STACK

Python · LangChain / LangGraph · Claude / GPT-4o API · Structured Outputs · NetworkX · ChromaDB / FAISS · FastAPI · Streamlit / React

SUGGESTED DATASETS

- Self-generated synthetic data using Faker and SDV libraries: customer profiles (200+), transaction ledger (500+ rows), simplified sanctions list (100+ entries), and 5 prior SAR narrative templates. A sample data generation script is available in the hackathon GitHub repository (sksaha-hcl/hackathon-starter-kit).
- Kaggle: PaySim Synthetic Mobile Money Transactions (proxy dataset for realistic transaction volume and patterns)
- Kaggle: Elliptic Bitcoin Dataset (labelled transaction graph for typology pattern development and testing)

12-DAY SPRINT TIMELINE (JUL 6–16)

Days 1–2	Days 3–5	Days 6–8	Days 9–10	Days 11–12

Synthetic data generation, agent scaffolding & architecture design	Agentic evidence pipeline: KYC, transaction history & sanctions tool calls	Risk scoring engine with reasoning chain + typology detection	SAR narrative generator — FinCEN/FCA field mapping, citations & gap detection	Analyst Q&A UI, human-in-the-loop checkpoint, demo polish & presentation

DELIVERABLES

1. Working agentic pipeline: alert intake → evidence gathering → risk score → SAR draft (live or recorded demo on 2+ synthetic test cases)
2. Multi-turn analyst Q&A interface with source citations on every answer and graceful handling of missing data
3. SAR narrative output with all mandatory fields populated and every factual claim linked to an evidence source
4. Architecture diagram showing agent tools, data sources, handoff payloads, and human-in-the-loop approval checkpoint
5. Hallucination mitigation brief: factual grounding strategy, confidence-threshold design, and audit trail approach

JUDGING CRITERIA (TOTAL: 100%)

30% Reasoning quality & decision explainability	**25%** SAR compliance & hallucination mitigation	**20%** Agentic pipeline architecture & tool design	**15%** Analyst UX & human-in-the-loop design	**10%** Live demo & speed improvement quantified

BONUS CHALLENGES (OPTIONAL — EXTRA CREDIT)

+ Extend to full graph network visualisation of connected entities across 2+ hops
+ Support both FinCEN and FCA regulatory SAR formats generated in a single pipeline run
+ Build a false-positive auto-close engine that triages alerts before analyst assignment with auditable rationale
+ Add an orchestration dashboard showing live agent status, inter-agent messages, and a full case timeline